INNOMATICS RESEARCH LABS

RAG-Based Customer Support Assistant

with LangGraph & Human-in-the-Loop (HITL)

Technical Documentation | Internship Project

1. Introduction

1.1 What is Retrieval-Augmented Generation (RAG)?

Retrieval-Augmented Generation (RAG) is an AI architecture pattern that enhances Large Language Models (LLMs) by combining them with an external knowledge retrieval system. Instead of relying solely on knowledge baked into the model during training, RAG dynamically fetches relevant context from a curated knowledge base at query time, then passes that context along with the user query to the LLM for answer generation.

The three-phase RAG pipeline works as follows:

- Indexing Phase: Source documents are ingested, chunked into smaller passages, converted into dense vector embeddings, and stored in a vector database.
- Retrieval Phase: When a query arrives, it is similarly embedded, and a similarity search finds the most semantically relevant chunks from the vector store.
- Generation Phase: The retrieved chunks are injected into a prompt as context, and the LLM generates a grounded, accurate response.

Key Advantage of RAG

RAG solves the hallucination problem in LLMs. Because the model receives factual context from trusted
documents at inference time, it generates answers grounded in real, up-to-date information rather than
potentially outdated or fabricated training knowledge.

1.2 Why is RAG Needed?

Traditional LLMs suffer from critical limitations in enterprise settings:

Limitation	Problem	RAG Solution
Knowledge Cutoff	LLMs cannot answer about recent events or proprietary data.	RAG retrieves from live, updated knowledge bases.
Hallucination	LLMs can confidently generate incorrect answers.	Answers are grounded in retrieved factual chunks.
Context Window Limits	Feeding entire large documents into a prompt is not feasible.	Only the top-K relevant chunks are retrieved and used.
Lack of Domain Specificity	General LLMs lack deep knowledge of specific products or policies.	RAG is tuned to domain-specific PDF/document corpora.
Auditability	It is hard to trace why an LLM gave a specific answer.	RAG provides source documents for every retrieved answer.

1.3 Use Case Overview

This project implements a RAG-based Customer Support Assistant for a product or service company. The system processes internal documentation (product manuals, FAQs, policy documents) stored as PDFs and answers customer queries in natural language.

Core capabilities of the system:

- Accepts natural language questions from customers via CLI or Web UI.
- Retrieves relevant sections from a PDF knowledge base using semantic search.
- Generates contextual, accurate answers using an LLM guided by retrieved context.
- Detects intent and routes queries: simple queries are answered automatically; complex or ambiguous queries escalate to human agents via HITL.
- Orchestrates the full workflow using LangGraph, a graph-based state machine framework.

2. System Architecture

2.1 High-Level Architecture Overview

The system is composed of two major pipelines:

- Offline Ingestion Pipeline: Processes source PDFs into a searchable vector store (runs once or on document update).
- Online Query Pipeline: Handles live user queries end-to-end using retrieval, LLM generation, intent routing, and HITL escalation (runs per query).

2.2 Architecture Components

Layer	Component	Technology	Responsibility
User Interface	CLI / Web Frontend	Python CLI or Streamlit	Accepts user query, displays final answer
Document Ingestion	PDF Loader	LangChain PyPDFLoader	Loads and extracts text from PDF documents
Chunking	Text Splitter	RecursiveCharacterTextSplitter	Splits documents into overlapping chunks
Embedding	Embedding Model	OpenAI text-embedding-ada-002	Converts text chunks/queries to dense vectors
Vector Store	ChromaDB	ChromaDB (local/persistent)	Stores and indexes embeddings for similarity search
Retrieval	Retriever	LangChain VectorStoreRetriever	Fetches top-K relevant chunks for a query
LLM Processing	Language Model	GPT-4 / Claude 3 Sonnet	Generates answers from context + query prompt
Workflow Engine	Graph Orchestrator	LangGraph StateGraph	Controls the node-based execution flow
Intent Router	Routing Logic	Custom Python / LLM classifier	Classifies query intent and routes accordingly
HITL	Escalation Module	LangGraph interrupt / Human node	Escalates unresolvable queries to human agents

2.3 Data Flow: PDF to Answer

Offline Phase (Document Ingestion):

1. PDF documents are loaded from disk using PyPDFLoader.

2. Loaded documents are split into overlapping chunks (500 tokens, 50 overlap) by RecursiveCharacterTextSplitter.
3. Each chunk is passed through the embedding model to generate a 1536-dimensional vector.
4. Vectors and their associated metadata (source, page number, chunk text) are persisted in ChromaDB.

Online Phase (Query Handling):

5. User submits a query via CLI or Web UI.
6. The query is embedded using the same embedding model.
7. ChromaDB retrieves the top-K most similar chunks via cosine similarity search.
8. The LangGraph workflow is initiated with the query and retrieved chunks as input state.
9. The Processing Node builds a prompt and calls the LLM to generate an answer.
10. The Router Node evaluates the LLM response confidence score.
11. If confidence is high: the answer is returned to the user via the Output Node.
12. If confidence is low or query is complex: the HITL Escalation Node is triggered, pausing execution for human review.
13. Human agent reviews and provides a response, which is integrated back into the graph state and delivered to the user.

2.4 Technology Choices

Why ChromaDB?

- Open-source, lightweight, and runs locally without external service dependencies.
- Supports persistent storage, enabling the vector index to survive application restarts.
- Native integration with LangChain through the LangChain Chroma wrapper class.
- Supports metadata filtering, enabling source attribution alongside similarity search.
- Scalable for moderate document collections; can be swapped for Pinecone or Weaviate at larger scale.

Why LangGraph?

- Provides a stateful, graph-based execution model ideal for multi-step AI pipelines.
- Supports conditional edge routing, allowing dynamic branching based on system state.
- Built-in support for Human-in-the-Loop interruption using interrupt_before and interrupt_after hooks.
- Integrates natively with LangChain components, reducing boilerplate code.
- Enables persistent state across multiple turns, supporting multi-turn conversations.

LLM Choice

- Primary: GPT-4 Turbo — high accuracy, strong instruction following, excellent for complex queries.
- Alternative: Claude 3 Sonnet — cost-effective with strong reasoning for production deployments.
- Fallback: GPT-3.5-Turbo — for high-throughput, cost-sensitive scenarios.

2.5 Scalability Considerations

Concern	Current Approach	Scalable Approach
Large Documents	Single PDF loaded at startup	Asynchronous batch ingestion pipeline
High Query Load	Synchronous single-user CLI	FastAPI backend with async handlers + Redis queue
Vector Search Latency	ChromaDB local instance	Pinecone / Weaviate managed cloud vector DB
LLM Latency	Sequential LLM call per query	Streaming responses + caching frequent queries
Multiple Documents	Single knowledge base	Namespace-separated multi-tenant vector collections

3. Low-Level Design

3.1 Module-Level Design

Module 1: Document Processing Module

- Responsibility: Load PDF files from a specified directory and extract raw text pages.
- Input: File path to PDF document(s).
- Output: List of LangChain Document objects with page_content (text) and metadata (source, page).
- Technology: LangChain PyPDFLoader. Each page becomes a separate Document object.

Module 2: Chunking Module

- Responsibility: Split long document pages into smaller, semantically coherent text chunks.
- Strategy: RecursiveCharacterTextSplitter with chunk_size=500, chunk_overlap=50.
- Rationale: 500 tokens balances retrieval precision and context completeness. 50-token overlap prevents context loss at chunk boundaries.
- Output: List of Document chunks with inherited metadata plus chunk index.

Module 3: Embedding Module

- Responsibility: Transform text chunks and queries into dense vector representations.
- Model: OpenAIEmbeddings using text-embedding-ada-002 (1536-dimensional vectors).
- Batch processing: All chunks are embedded in a single batch API call during ingestion.

Module 4: Vector Storage Module

- Responsibility: Store, index, and retrieve embedded vectors with metadata.
- Technology: ChromaDB with persistent storage at ./chroma_db directory.
- Collection schema: Each document stores vector, text content, and metadata (source, page, chunk_id).
- Operations: upsert (ingestion), similarity_search_with_score (retrieval).

Module 5: Retrieval Module

- Responsibility: Given a query embedding, find the top-K most similar document chunks.
- Method: Cosine similarity search via ChromaDB retriever.
- Configuration: k=4 (retrieves 4 chunks by default). Configurable via retriever search_kwargs.

- Returns: List of (Document, score) tuples where score is cosine similarity (0-1).

Module 6: Query Processing Module (LLM Node)

- Responsibility: Build a structured prompt from retrieved context and generate an answer using the LLM.
- Prompt Template: System prompt defines the assistant role; user prompt includes retrieved chunks and the user query.
- Confidence scoring: LLM is prompted to return a confidence score (0-1) alongside its answer.
- Output: Structured response containing answer text, confidence score, and sources used.

Module 7: Graph Execution Module (LangGraph)

- Responsibility: Orchestrates the entire query pipeline as a directed state graph.
- Framework: LangGraph StateGraph with TypedDict state definition.
- Nodes: input_node, retrieve_node, process_node, route_node, output_node, hitl_node.
- Edges: Conditional edges from route_node to either output_node or hitl_node based on confidence.

Module 8: HITL Module

- Responsibility: Pause graph execution, notify a human agent, collect response, and resume graph flow.
- Trigger: Confidence score below 0.65, escalation keywords, or no relevant chunks found.
- Mechanism: LangGraph interrupt() pauses the graph; human input is fed back as graph state update.

3.2 GraphState Data Structure

GraphState (TypedDict) — Central State Object

```
query: str                  — The original user question
retrieved_chunks: List[Doc]   — Top-K chunks from ChromaDB
context: str                — Formatted string of chunk contents for the prompt
answer: str                 — Generated answer from the LLM
confidence: float           — LLM confidence score (0.0 to 1.0)
sources: List[str]          — Source document names and page numbers
route: str                  — Routing decision: 'answer' | 'escalate' | 'error'
human_response: Optional[str] — Response from human agent if escalated
final_answer: str           — The answer delivered to the user
```

3.3 LangGraph Workflow Design

Node	Type	Responsibility	Next Node(s)
input_node	Start Node	Receives user query, initializes graph state	retrieve_node
retrieve_node	Processing Node	Embeds query, retrieves top-K chunks from ChromaDB	process_node
process_node	Processing Node	Builds prompt, calls LLM, parses answer + confidence	route_node
route_node	Conditional Node	Evaluates confidence score and routes accordingly	output_node OR hitl_node
output_node	Terminal Node	Formats and delivers final answer to user	END
hitl_node	Interrupt Node	Pauses graph, queues for human review, integrates response	output_node

3.4 Conditional Routing Logic

- Route to output_node (Auto-Answer) when:
  - Confidence score >= 0.65 AND
  - At least 1 relevant chunk was retrieved AND
  - No escalation keywords detected in the query.

- Route to hitl_node (Escalate to Human) when:
  - Confidence score < 0.65 (LLM uncertain), OR
  - Zero chunks retrieved (query outside knowledge base scope), OR
  - Query contains escalation keywords: 'refund', 'legal', 'complaint', 'urgent', 'lawsuit', OR
  - Query is identified as emotionally charged or requiring personalized judgment.

- Route to error handler when:
  - LLM API call fails or returns malformed response.
  - ChromaDB is unavailable or returns an error.

3.5 HITL Design

Phase	Action	Implementation Detail
Trigger Detection	route_node identifies escalation condition	Confidence < 0.65 or escalation keyword match
Graph Pause	LangGraph interrupt() is called before hitl_node	Execution halts; state is serialized and queued
Agent Notification	System logs escalation event with query + context	Console alert / webhook / email notification
Human Review	Agent views original query and retrieved chunks	Admin UI or CLI prompt displays full context
Response Input	Human provides corrected or supplemental answer	Input stored in human_response state key
Graph Resume	Graph state updated with human_response	StateGraph.update_state() called with human input
Delivery	output_node delivers human response to user	Response tagged as Reviewed by Support Agent

3.6 API / Interface Design

Input Format (JSON)

```
{ "query": "string — the customer question",
  "session_id": "string — optional, for multi-turn conversations",
  "knowledge_base": "string — optional, defaults to default" }
```

Output Format (JSON)

```
{ "answer": "string — the generated or human-reviewed answer",
  "confidence": "float — confidence score 0.0 to 1.0",
  "sources": ["filename.pdf, page 3", ...],
  "status": "auto_answered | escalated | error",
  "session_id": "string" }
```

3.7 Error Handling

Error Condition	Detection	Handling Strategy
PDF Load Failure	FileNotFoundError or corrupted PDF	Log error, skip file, alert administrator
Embedding API Failure	OpenAI API timeout or rate limit	Retry with exponential backoff (3 attempts)

Error Condition	Detection	Handling Strategy
ChromaDB Unavailable	Connection error on similarity search	Return error state, trigger HITL escalation
No Relevant Chunks Found	Empty retrieval or all scores < 0.3	Set confidence=0, route to HITL escalation
LLM API Failure	HTTP 500 or timeout from LLM provider	Retry once, then escalate to HITL node
Malformed LLM Response	JSON parse error or missing confidence field	Default confidence=0, route to HITL node
Graph State Corruption	Missing required keys in TypedDict state	Catch KeyError, reset state, return error response

4. Workflow Explanation

4.1 LangGraph Usage

LangGraph is a library built on top of LangChain that enables stateful, multi-actor applications using a directed graph structure. In this system, the entire query-answer lifecycle is modeled as a LangGraph StateGraph, where each node performs a discrete function and edges define the flow of execution.

Key LangGraph concepts used in this project:

- StateGraph: The container that holds all nodes and edges. Initialized with the GraphState TypedDict schema.
- Nodes: Python functions that take the current state as input and return an updated state dictionary.
- Edges: Connections between nodes. Can be direct (always execute) or conditional (execute based on state).
- Conditional Edges: add_conditional_edges() maps state values to target nodes, enabling dynamic routing.
- Interrupt: LangGraph interrupt mechanism pauses graph execution at a specified node for HITL workflows.
- Checkpointing: LangGraph can serialize and restore graph state, enabling conversation memory across sessions.

4.2 Node Responsibilities

Node	Input State Keys	Output State Keys	Core Logic
input_node	query	query (validated)	Sanitize and validate user query string
retrieve_node	query	retrieved_chunks, context	Embed query, run similarity search, format chunks
process_node	query, context	answer, confidence, sources	Build prompt, call LLM, parse structured response
route_node	confidence, retrieved_chunks, query	route	Apply routing logic, set route to answer or escalate
output_node	answer OR human_response, sources	final_answer	Format and return final response to user
hitl_node	query, context, answer	human_response	Pause execution, await human input, resume with response

4.3 State Transitions

Normal (Happy Path) State Transition:

14. START -> input_node: Graph is initialized with the user query.
15. input_node -> retrieve_node: Query is validated; retrieval begins.
16. retrieve_node -> process_node: Chunks retrieved; LLM processing begins.
17. process_node -> route_node: Answer and confidence score generated; routing decision made.
18. route_node -> output_node: (confidence >= 0.65) Answer is delivered. Graph reaches END.

Escalation (HITL) State Transition:

19. process_node -> route_node: LLM generates low-confidence answer or detects escalation trigger.
20. route_node -> hitl_node: Route is set to escalate. Graph execution is interrupted.
21. [PAUSE] Human agent reviews query, context, and draft answer. Provides corrected response.
22. hitl_node -> output_node: Human response is injected into state. Graph resumes.
23. output_node -> END: Human-reviewed answer is delivered with Escalated status tag.

5. Conditional Logic

5.1 Intent Detection

Intent detection determines how a user query should be handled. The system uses a two-stage intent classifier:

Stage 1: Keyword-Based Pre-filter

A fast pattern-matching pass checks for escalation trigger words before the LLM is called:

- Escalation keywords: refund, legal, lawsuit, complaint, urgent, cancel subscription, data breach.
- Any query containing these keywords is immediately routed to HITL, bypassing LLM generation.
- This ensures sensitive topics are never auto-answered without human oversight.

Stage 2: LLM Confidence-Based Routing

Confidence Range	Interpretation	Action
0.85 - 1.00	High confidence — LLM found strong supporting context	Auto-answer immediately
0.65 - 0.84	Moderate confidence — LLM found partial context	Auto-answer with source citations
0.40 - 0.64	Low confidence — LLM uncertain, weak context match	Escalate to human agent (HITL)
0.00 - 0.39	Very low / no context found	Escalate to human agent (HITL)

5.2 Routing Decisions

The routing layer is implemented as a Python function passed to add_conditional_edges(). It evaluates the GraphState and returns a string key mapping to the target node:

- Returns 'answer' (-> output_node): confidence >= 0.65 AND no escalation keywords AND chunks retrieved > 0.
- Returns 'escalate' (-> hitl_node): any escalation condition is met.
- Returns 'error' (-> error_node): critical system failure detected (LLM unavailable, DB error).

6. HITL Implementation

6.1 Role of Human Intervention

The Human-in-the-Loop (HITL) module serves as the safety net of the customer support system. It ensures that queries which the AI cannot confidently or appropriately answer are handled by a human support agent, maintaining service quality and preventing harmful auto-responses.

HITL is essential in the following scenarios:

- Factual uncertainty: The LLM cannot find sufficient supporting evidence in the knowledge base.
- Sensitive topics: Legal, financial, or emotionally charged queries require human judgment.
- Novel queries: Customer questions about scenarios not covered in the documentation.
- Compliance requirements: Regulated industries may mandate human review for certain query types.

6.2 Benefits and Limitations

Aspect	Benefits	Limitations
Quality	Human-reviewed answers are highly accurate and contextual	Increases response latency for escalated queries
Safety	Prevents harmful or incorrect auto-responses on sensitive topics	Requires trained support staff availability
Trust	Users receive assurance that complex issues are human-handled	Human agents may be inconsistent in response quality
Learning	Escalation data can be used to fine-tune the LLM over time	Feedback loop requires additional engineering effort
Scalability	Handles edge cases gracefully without system crashes	HITL does not scale linearly with query volume

7. Challenges & Trade-offs

7.1 Retrieval Accuracy vs. Speed

The vector similarity search is the performance bottleneck in the online pipeline. Increasing the number of retrieved chunks (higher K) improves answer completeness but increases LLM prompt size and processing time. Using a re-ranker model (e.g., Cohere Rerank) can improve precision but adds additional API call latency. The chosen default of K=4 balances retrieval coverage with acceptable latency for most queries.

7.2 Chunk Size vs. Context Quality

Smaller chunks (200 tokens) allow more precise retrieval but may cut off important context mid-sentence. Larger chunks (1000 tokens) preserve context integrity but may retrieve irrelevant peripheral content that dilutes the prompt. The chosen 500-token chunk size with 50-token overlap represents an empirically validated middle ground, though optimal values are domain-dependent and should be tuned against a representative query set.

7.3 Cost vs. Performance

Using GPT-4 Turbo delivers the best answer quality but incurs higher API costs per query. For high-volume deployments, GPT-3.5-Turbo or a self-hosted open-source model (Llama 3, Mistral) may reduce per-query costs, with the trade-off of lower quality on complex queries. A tiered strategy — using GPT-3.5-Turbo for simple queries and GPT-4 for HITL-flagged queries — optimizes the cost-quality balance.

7.4 Summary of Key Trade-offs

Trade-off	Option A	Option B	Recommendation
Vector DB	ChromaDB (local, free)	Pinecone (managed, paid)	ChromaDB for dev; Pinecone for production
Embedding Model	OpenAI ada-002 (paid, high quality)	Sentence-Transformers (free, local)	OpenAI for production; SBERT for cost-sensitive
HITL Trigger	Rule-based (fast, predictable)	LLM-based classification (flexible)	Hybrid: rules first, LLM classifier as fallback
Memory	No memory (stateless per query)	LangGraph persistence (multi-turn)	Stateless for MVP; add memory for production

8. Testing Strategy

8.1 Testing Approach

The system is tested across three layers:

- Unit Testing: Each module (chunker, embedder, retriever, LLM caller, router) is tested independently with mock inputs and expected outputs.
- Integration Testing: The full LangGraph pipeline is run end-to-end with a controlled test knowledge base to verify correct state transitions and routing decisions.
- User Acceptance Testing (UAT): Domain experts review system responses against a curated set of 30+ representative customer queries, evaluating answer accuracy, completeness, and appropriate routing.

8.2 Sample Test Queries

Query	Expected Route	Expected Behavior
What is your return policy?	Auto-Answer	Retrieves return policy chunk; high confidence response
How do I reset my password?	Auto-Answer	Retrieves account help chunk; step-by-step answer
My account was hacked — I need help NOW	Escalate (HITL)	Escalation keyword urgent; routes to human agent
Can I get a refund for last month?	Escalate (HITL)	Escalation keyword refund; routes to human agent
What is the capital of France?	Escalate (HITL)	No relevant chunks found; confidence near 0; HITL
What are your business hours?	Auto-Answer	High-confidence retrieval from FAQ document
I want to file a legal complaint	Escalate (HITL)	Escalation keyword legal; immediate HITL routing

9. Future Enhancements

9.1 Multi-Document Support

The current system supports a single PDF knowledge base. Future versions will support ingestion of multiple documents across different formats (PDF, DOCX, HTML, Confluence pages) with namespace-separated ChromaDB collections per department or product line.

9.2 Feedback Loop & Continuous Learning

User ratings (thumbs up/down) on answers will be logged to a feedback database. HITL escalations and human corrections will be stored as high-quality training examples. Periodic fine-tuning of the LLM on domain-specific Q&A pairs derived from escalation logs will progressively reduce the escalation rate over time.

9.3 Conversation Memory Integration

LangGraph checkpoint persistence will be leveraged to maintain conversation history across multiple turns within a session. The retrieval step will incorporate previous query context to better resolve follow-up questions.

9.4 Deployment Architecture

The system will be containerized using Docker and deployed on a cloud platform (AWS / GCP). The query pipeline will be exposed as a REST API via FastAPI. A Streamlit or React frontend will provide the customer-facing chat interface. ChromaDB will be replaced with Pinecone or Weaviate for production-grade scalability.

Enhancement	Priority	Effort	Impact
Multi-document ingestion	High	2 weeks	Extends system to full enterprise knowledge base
Feedback loop & fine-tuning	High	3 weeks	Reduces HITL escalations; improves accuracy over time
Conversation memory	Medium	1 week	Handles follow-up questions naturally
Docker + API deployment	Medium	1 week	Enables production use by real customers
Re-ranking model integration	Low	1 week	Improves retrieval precision for ambiguous queries
Analytics dashboard	Low	2 weeks	Visibility into query patterns and escalation rates

10. Key Design Decisions

10.1 Chunk Size Selection

After evaluating retrieval performance across chunk sizes of 200, 300, 500, and 800 tokens on a sample document set, 500 tokens with 50-token overlap was chosen. At 200 tokens, retrieved chunks often lacked sufficient context to answer questions fully. At 800 tokens, the LLM prompt exceeded context window limits and retrieval precision dropped. 500 tokens consistently produced the best balance of answer completeness and retrieval precision.

10.2 Embedding Strategy

OpenAI text-embedding-ada-002 was selected for several reasons: it produces 1536-dimensional vectors with strong semantic fidelity, is cost-effective at $0.0001 per 1K tokens, and is natively supported by both LangChain and ChromaDB. The same model is used for both indexing (document chunks) and querying (user queries), ensuring the embedding space is consistent and similarity comparisons are valid.

10.3 Retrieval Approach

Dense retrieval via cosine similarity was chosen over sparse retrieval methods (BM25/TF-IDF) because customer support queries are often phrased differently from the exact wording in documentation. Dense embeddings capture semantic meaning rather than exact keyword matches, making them far more robust to paraphrasing and synonym use. Hybrid retrieval (combining dense and sparse) is planned as a future enhancement.

10.4 Prompt Design Logic

The LLM prompt is structured with three key components: (1) a role definition instructing the model to act as a helpful customer support assistant, (2) the retrieved context chunks clearly labeled with source and page number, and (3) an explicit instruction to return a JSON object containing both the answer and a confidence score. This structured output format enables deterministic parsing and confidence-based routing without additional LLM calls.

Appendix: Evaluation Criteria Summary

Criteria	Weight	Key Evaluation Points
HLD Quality	20%	Architecture completeness, component coverage, diagram clarity, technology justification
LLD Depth	20%	Module specifications, data structures, workflow design, error handling completeness
Technical Documentation	25%	Clarity of explanation, design reasoning, trade-off analysis, testing strategy
Concept Application	20%	Correct application of RAG, LangGraph, ChromaDB, HITL, conditional routing
Clarity & Presentation	15%	Document structure, professional formatting, readable diagrams and tables

End of Technical Documentation